# ATS Automation - Deployment Guide

## Per Agent Machine Requirements

### Minimum Hardware

| Requirement | Specification |
|---|---|
| OS | Windows 10/11 or macOS 12+ |
| RAM | 8GB minimum |
| Storage | 10GB free |
| CPU | Intel i5 / AMD Ryzen 5 or better |
| Display | 1920x1080 or higher |

### Software Requirements

| Requirement | Version |
|---|---|
| Python | 3.10+ |
| Chrome | Latest stable |
| Edge | Latest stable (optional) |
| Network | Access to all client portals |

### Permissions Required

```
- Install Chrome Extension
- Run Python scripts
- Access Downloads folder
- (Optional) Admin rights for autostart
```

---

## Installation Steps

### Step 1: Prepare Environment

```
# Create project folder
mkdir ~/ats-automation
cd ~/ats-automation

# Create virtual environment
python3 -m venv venv
source venv/bin/activate  # Windows: venv\Scripts\activate
```

```
# Install dependencies
pip install -r requirements.txt
```

### Step 2: Install Playwright

```
# Install Playwright
pip install playwright
playwright install chromium

# Install system dependencies (Linux/macOS)
playwright install-deps
```

### Step 3: Install Chrome Extension

1. Open Chrome → `chrome://extensions/`
2. Enable **Developer mode** (top right)
3. Click **Load unpacked**
4. Select: `~/ats-automation/chrome-extension/`
5. Verify extension loads without errors

### Step 4: Configure Client

```
# Copy client config template
cp config/client-template.json config/your-client-name.json

# Edit with client-specific details
nano config/your-client-name.json
```

### Step 5: Test Installation

```
# Run test script
python tests/test_installation.py

# Expected output:
# ✓ Python version OK
# ✓ Playwright installed
# ✓ Chrome extension loaded
# ✓ Config files valid
```

---

## Initial Setup Checklist

### For IT/Admin

- [ ] Python 3.10+ installed
- [ ] Chrome/Edge installed
- [ ] Network access to all client portals verified

- [ ] Extension installation permissions granted
- [ ] Downloads folder accessible

### For Team Lead

- [ ] Client config file created
- [ ] DOM selector mapping completed
- [ ] Test accounts identified
- [ ] Agent training scheduled

### For Pilot Agents

- [ ] Installation walkthrough completed
- [ ] First automation demonstrated
- [ ] Hotkey shortcuts documented
- [ ] Emergency fallback explained

---

## Configuration Files

### Client Config Structure

```json
{
  "client_name": "flyland",
  "display_name": "Flyland Recovery",
  "systems": {
    "ctm": {
      "url_pattern": "*.calltrackingmetrics.co",
      "enabled": true
    },
    "salesforce": {
      "url_pattern": "*.salesforce.com",
      "enabled": true,
      "filters": {
        "owner": "flyland-agents"
      }
    }
  },
  "automations": {
    "ctm_sf_auto_access": {
      "enabled": true,
      "delay_ms": 500
    },
    "auto_note_generator": {
      "enabled": true,
      "confidence_threshold": 0.7
    }
  },
  "logging": {
```

```
    "level": "INFO",
    "save_to_file": true
  }
}
```

### Field Mapping Config

```
{
  "ctm_to_sf": {
    "phone": {
      "ctm_selector": "#caller-phone",
      "sf_selector": "#phone"
    },
    "disposition": {
      "ctm_selector": "#disposition-code",
      "sf_selector": "#Status"
    }
  }
}
```

### Automation Templates

```
{
  "reply_templates": {
    "unqualified": [
      "Thank you for calling. Unfortunately, we are unable to help at this time.",
      "We appreciate your interest. Unfortunately, we cannot assist at this
moment."
    ],
    "callback": [
      "Thank you for your patience. We will call you back at {time}.",
      "A specialist will contact you within 24 hours."
    ]
  }
}
```

---

## Running Automations

### Manual Start

```
# Start specific automation
python -m playwright.c tm_sf_auto_access

# Start all enabled automations
python main.py start
```

```
# Start with specific client config
python main.py start --config flyland
```

## Background Service

```
# Install as system service (optional)
# Windows: Use Task Scheduler
# macOS: Use launchd
# Linux: Use systemd

# Or use simple autostart
# Add to .bashrc or startup folder
python ~/ats-automation/main.py start
```

## Hotkey Controls

| Hotkey | Action |
|---|---|
| Ctrl+Shift+A | Toggle CTM-SF Auto-Access |
| Ctrl+Shift+N | Generate Auto-Note |
| Ctrl+Shift+S | Sync Wrap-Up |
| Ctrl+Shift+D | Disable All |
| Ctrl+Shift+L | Open Logs |

---

# Troubleshooting

## Common Issues

### Issue: Extension not loading

```
Solution:
1. Check chrome://extensions/ for errors
2. Reload extension
3. Check manifest.json syntax
4. Verify all files present
```

### Issue: Playwright fails to launch

```
Solution:
1. Run: playwright install chromium
2. Run: playwright install-deps
3. Check Python version (3.10+)
4. Try: pip install --upgrade playwright
```

### Issue: Can't find DOM elements

```
Solution:
1. Open browser DevTools (F12)
2. Inspect element
3. Update selector in config
4. Run test mode for debugging
```

### Issue: Automation too slow

```
Solution:
1. Check polling interval in config
2. Reduce unnecessary DOM reads
3. Increase delay between actions
4. Close unused browser tabs
```

## Debug Mode

```
# Enable debug logging
python main.py start --debug

# See detailed DOM operations
python -m playwright.c tm_sf_auto_access --verbose
```

## Log Locations

```
~/ats-automation/logs/
├── automation.log      # Main log
├── errors.log          # Errors only
├── performance.log     # Timing metrics
└── chrome_extension/
    └── extension.log   # Extension debug
```

---

# Uninstallation

## Remove Chrome Extension

1. Open `chrome://extensions/`
2. Find "ATS Automation"
3. Click Remove

## Remove Python Scripts

```
# Deactivate virtual environment
deactivate

# Remove project folder
rm -rf ~/ats-automation
```

```
# Optionally remove Python
# (may affect other applications)
```

---

## Support Contacts

| Role | Contact | Response Time |
|---|---|---|
| Technical Lead | [TBD] | 4 hours |
| Automation Dev | [TBD] | 8 hours |
| General Support | [TBD] | 24 hours |

### Issue Reporting

```
1. Check logs first (logs/automation.log)
2. Note exact error message
3. Note your OS and Chrome version
4. Describe steps to reproduce
5. Send to support with above info
```

---

## Maintenance

### Weekly Tasks

- [ ] Review error logs
- [ ] Check automation success rate
- [ ] Gather agent feedback

### Monthly Tasks

- [ ] Update Chrome Extension (if new version)
- [ ] Review and update DOM selectors
- [ ] Backup configuration files
- [ ] Review NLP model performance

### Quarterly Tasks

- [ ] Full system health check
- [ ] Retrain NLP models (if needed)
- [ ] Plan improvements based on usage

---

# Security Considerations

## What Automations Can Access

- Only data visible in agent's browser
- Only during active session
- Only on permitted websites

## What Automations Cannot Access

- Passwords or login credentials
- Data outside browser
- Internal company systems (no API)

## Best Practices

1. Never share automation logs externally
2. Keep config files local (not in cloud)
3. Restart browser daily (clears session data)
4. Report any unusual automation behavior

---

*Last updated: March 2026*